RESEARCH UPDATE · RB-DERIV

Retrieval-of-Thought: a memory bank of multi-solution reasoning

How a Qwen3-8B teacher's solutions become memory and a graph, and how that memory is retrieved and injected to help a Qwen3-1.7B student solve test problems

Atique Ahmad · October 2026

THE QUESTION

What the experiment tests

Does handing a small model someone else's reasoning strategy help it solve a problem it can't crack alone? And is it the **retrieval** that helps, or would any worked example do?

8B

Teacher: Qwen3-8B-AWQ, thinking on, writes the memory

1.7B

Student: Qwen3-1.7B, thinking on, uses the memory

192

RB-DERIV problems: 16 technique families × 12, exact integer answers

144 / 48

Train / test: 9 and 3 per family, frozen in split.json

THE PIPELINE

Six stages, each writing a file the next one reads

1 · GENERATE

Teacher solves each train problem 5 ways

`teacher_solutions`

2 · VALIDATE

Exact match against the gold integer

`…_validated`

3 · BUILD MEMORY

Problem → solution → approach → subtasks

`memory_bank`

4 · BUILD GRAPHS

Embed nodes, add structural and semantic edges

`thought_graph`

5 · RETRIEVE + INJECT

Find memory for a test problem, put it in the prompt

`eval_<arm>`

6 · REPORT

Accuracy, tokens, latency, per family

`summary.md`

Stages 1–4: offline, run once on the 144 train problems **Stages 5–6:** online, once per test question and arm

STAGE 1 · GENERATE

Five different solutions for every training problem

720 teacher calls

144 train problems × 5 independent samples, batched through vLLM

Forced diversity

Temperature 0.8, plus a different strategy hint per sample, so the five reach the answer by different routes

Fixed output format

The teacher must write an APPROACH line and numbered STEP lines; a regex splits them into the approach and the mini-subtasks

Reusable steps

Rule in the prompt: each step must be one sub-task that could be followed on a different problem of the same type

```
APPROACH: <technique, ≤12 words>
STEP 1: <one reasoning action>
STEP 2: <one reasoning action>
STEP k: …
ANSWER: <integer>
```

sample	hint appended to the problem
s0	none
s1	a different strategy than the obvious one
s2	look for periodicity, cycle, modular structure
s3	derive a closed form, avoid iteration
s4	use counting, symmetry or invariance

EXAMPLE · PROBLEM Q0009, FAMILY LCG_CLOSED

One problem, several solutions, each split into subtasks

```
Let x(0)=3 and x(k+1) = (4*x(k) + 9) mod 997. What is x(1000000000000001)? gold = 866
```

entry q0009_s0 · answer 866 ✓

Approach: solve linear recurrence using closed form and modular exponentiation

1. Express recurrence as x(k) = (6·4^k + 994) mod 997
2. Compute 4^1000000000000001 mod 997 with Fermat's little theorem
3. Find n mod 996 to simplify the exponent
4. Calculate 4^29 mod 997 by successive squaring
5. Substitute into the closed form

entry q0009_s1 · answer 866 ✓

Approach: closed-form solution for linear recurrence and modular exponentiation

1. Express x(n) = 6·4^n – 3 mod 997
2. Apply Fermat: compute 4^N mod 997 via N mod 996
3. N = 10^15 + 1 mod 996 = 29
4. Compute 4^29 mod 997 = 311
5. (6·311 – 3) mod 997 = 866

STAGE 2 · VALIDATE

Every solution is checked against the gold integer

720
solutions generated

651
correct (90.4%), these enter the memory bank

41
parsed, but wrong answer

28
truncated at the token cap, unparseable

Nothing is deleted

Correctness, truncation and agreement with the majority answer are stored as metadata. Filtering is a separate, configurable choice in the next stage.

Teacher cost per sample

Mean 4,489 output tokens (3,878 of them thinking), 43.6 s latency. Paid once, offline.

STAGE 3 · BUILD MEMORY

The memory bank: one entry per verified solution


```
{
"entry_id": "q0009_s1",
"problem_id": "q0009",
"group": "lcg_closed",
"problem": "Let x(0)=3 and …",
"approach": "Use closed-form …",
"subtasks": ["Express x(n) …",
"Apply Fermat …", …],
"final_answer": 866,
"correct": true
}
```


651
entries

141
of 144 train problems covered

3.76
subtasks per entry

112 problems kept all five solutions; 29 kept one to four.

Admission rules

- Correct solutions only
- Between 2 and 12 subtasks
- Train problems only: the build raises an error if a test id appears

STAGE 4 · BUILD GRAPHS

Two graphs are built over the same memory bank

	Approach graph (arms C, D)	RoT step graph (arm F)
Nodes	651 approach + 2,450 subtask = 3,101	2,450 step nodes in 651 templates
Structural edges	belongs_to: subtask → its approach	sequential: step i → step i+1, 1,799
Semantic edges	similar_to, cosine ≥ 0.82, ≤ 8 per node: 24,006	cosine ≥ 0.85, uncapped: 598,636
Linking rule	never within one problem	never within one solution
Embedder	nomic-embed-text-v1, 768-d	jina-embeddings-v2-small, 512-d
Node text	approach; subtask with its approach	"approach | step"
Used for	finding similar problems' whole solutions	a walk that assembles a new template

Both are plain JSON adjacency lists plus a NumPy matrix of vectors: no graph library, no database.

STAGE 4 · ROT STEP GRAPH

How the step graph is built, in four passes

1 · Nodes

Each subtask of each bank entry becomes one node: 2,450 nodes, tagged with problem id, step index and family

→

2 · Sequence

Link step i to step i+1 inside each solution: 1,799 sequential edges

→

3 · Embed

Embed "approach | step" with jina-v2-small into 512-d, L2-normalised vectors

→

4 · Similarity

Link steps from different solutions with cosine ≥ 0.85: 598,636 semantic edges

244
semantic neighbours per node on average (max 704)

651
step-0 nodes, the possible entry points of a walk

47%
of semantic edges cross between families

~40 s
to build on the GPU; identical edge counts on CPU

STAGE 5 · RETRIEVAL FOR ARMS C AND D

Solution-level retrieval: nearest problems, top 3

1 · Embed
Test problem with the "search_query:" prefix

→

2 · Score
Cosine against all 651 entries' problem vectors

→

3 · Guard
Drop the same id and the same normalised text

→

4 · Top-k
Keep the 3 best entries

→

5 · Render
Format by mode and add to the user turn

mode	shown to the student
full_solution (C)	source problem + approach + all steps
approach_subtasks (D)	approach + numbered steps
approach	the approach line only
subtasks	the steps only

```
user turn:
Here is how similar problems were solved. The
approaches may or may not apply, judge that
yourself.
{3 retrieved memories}
Now solve this problem. …
```

STAGE 5 · RETRIEVAL FOR ARM F (ROT)

Step-level retrieval: a walk that assembles a template

#	stage	rule	why
1	Filter	keep steps of the query's family; drop the query's own problem	about 153 candidates left
2	First step	argmax 0.75·cos + 0.25·[step index is 0]	start at a real solution start
3	Next step	argmax 0.3·cos + 0.7·[sequential child]	follow a coherent procedure
4	Halt	cos < 0.8 and score < 0.8, no successor, or 8 steps	stop when nothing fits
5	Output	"Step 1: … Step n: …" as one template	may cross several solutions

Consequence of the weights

A sequential child scores 0.7 + 0.3·cos, at least 0.7; a semantic neighbour at most 0.3. So the walk follows one solution to its end, then jumps to another.

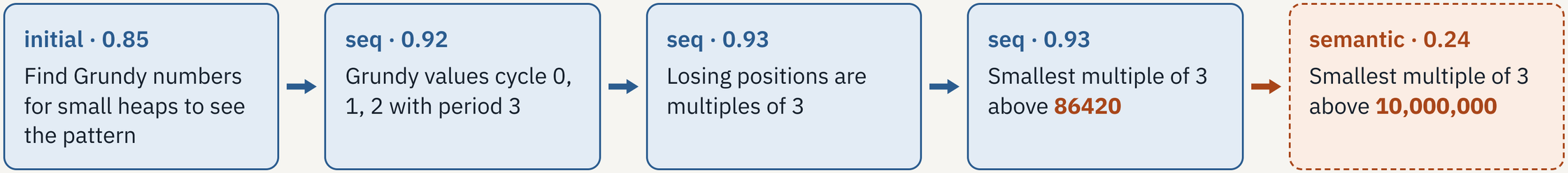
Measured on 48 test problems

Mean 6.98 steps from 2.85 source solutions; 42 of 48 templates crossed solutions. Edges taken: 196 sequential, 91 semantic.

WORKED EXAMPLE · TEST PROBLEM Q0042, FAMILY GRUNDY_124

One real walk through the step graph

Players alternately remove 1, 2 or 4 stones; whoever cannot move loses. Smallest losing heap size greater than 777777? **Gold 777780.** 124 candidate steps after the family filter.



Steps 1–4: one solution, q0046_s3. **Step 5:** jumps to q0038_s0, then the walk halts (below threshold).

The method is right, but the numbers **86420** and **10,000,000** come from other problems. The student still answered 777780 correctly (user-turn run).

STAGE 5 · INJECTION

Same template, two places to put it

Thinking Intervention (the paper's method)

Template is pre-filled inside an open <think> in the assistant turn; the model continues it as its own reasoning

```
assistant:
<think>I need to follow the given steps … The
template is. Step 1: … Step n: … I will now solve
the problem starting from the step 1. Using step 1
```

16.7% accuracy

User-turn injection (our ablation)

Template is reference material in the user turn; the model starts its own <think> and may reject it

```
user:
You are given a template to solve the problem. Use
the given steps if applicable … Step 1: … Step n: …
Problem: …
```

81.2% accuracy

Templates were byte-identical in both runs (48 of 48). The baseline is 75.0%.

RESULTS · QWEN3-1.7B ON 48 TEST PROBLEMS

Accuracy by arm

arm	what the prompt adds	acc	Δ pp	trunc
A · Baseline	nothing	75.0%	—	9
B · Random memory	3 random bank entries (control)	75.0%	+0.0	7
C · Full solution	3 retrieved full solutions	72.9%	−2.1	8
D · Approach + subtasks	3 retrieved approaches and steps	75.0%	+0.0	4
E · Gated	D's memory, only where A gave no answer	**85.4%**	**+10.4**	3
F · RoT, Thinking Intervention	walked template inside <think>	16.7%	−58.3	0
F · RoT, user turn	same template in the user turn	**81.2%**	**+6.2**	1
F · shuffled control	another problem's template	75.0%	+0.0	1

RoT memory bank · 14 · one problem = 2.08 pp · F rows: reference prompt, 15,360-token cap · results/*/summary.md

THE CONTROL

Retrieval is doing real work, not just "any plan helps"

+6.2 pp

Reference prompt, 15,360 cap: retrieved template 81.2% vs shuffled 75.0%

+10.4 pp

Project prompt, 8,192 cap: retrieved template 79.2% vs shuffled 68.8%

The shuffled template is one assembled for a **different** test problem: same machinery, same length, same crossing rate. A wrong template is worth nothing, or costs accuracy.

COST

Tokens, latency and memory footprint

arm	acc	in tok	think tok	lat s
A · Baseline	75.0%	173	2,755	19.0
C · Full solution	72.9%	719	2,247	15.9
D · Approach + steps	75.0%	598	2,754	19.2
E · Gated	85.4%	282	4,103	28.9
F · TI (exact)	16.7%	448	524	3.0
F · user turn (exact)	81.2%	378	3,579	29.5
F · user turn (project)	79.2%	411	2,726	17.8

Memory on disk

Bank 0.65 MB + vectors 2.0 MB. Approach graph 2.2 MB + 9.5 MB. RoT graph 28.6 MB + 5.0 MB.

Memory in the prompt

+205 to +546 input tokens per question over the baseline's 173

Building memory

720 teacher calls, ~40–70 min batched on one RTX 5070 Ti, once

RoT memory bank · 16 · student means over finished replies · latency = student generation per question

REPRODUCING THE PAPER

Under Thinking Intervention, the 1.7B stops solving

A typical complete generation in the exact reproduction:

```
… to 8, I will now solve the problem accordingly.
Therefore, the final answer is: $\boxed{12}$.
```

48 of 48 answers are in the right \boxed format, so parsing is not the cause.

paper's claim	measured	verdict
fewer tokens	2,755 → 524	reproduced
lower latency	19.0 → 3.0 s	reproduced
same accuracy	75.0 → 16.7%	not reproduced
retrieval transfers	+6.2 pp vs shuffled	supported
TI transfers	71 pp swing on site	not supported

Consistent with our earlier phase 3B finding: <think> injection did nothing on a 1.7B, while prompt injection helped.

LIMITATIONS

What these results do not show yet

gap	why it matters	fix
n = 48, no significance test	one problem is 2.08 pp; no CIs or McNemar yet	free, offline
single seed	run-to-run variance is not separated from effects	3× the runs
gold family tag in F	the 81.2% uses an advantage C/D did not have	48 calls
instance-bound steps	37% of steps carry a 4+ digit constant vs 0.2% in the paper	one teacher pass
easier benchmark	baseline traces ~2,766 tokens vs ~10k on AIME	dataset prep
one student size	TI might work at 4B or 8B, as in the paper	GPU time

Q&A · BUILDING MEMORY

Short answers, part 1

How did you create the memory?

Qwen3-8B solved each of 144 train problems 5 times in APPROACH / STEP / ANSWER format. 651 correct solutions became bank entries.

Why several solutions per problem?

Different routes give retrieval more strategies to match. Temperature 0.8 and per-sample strategy hints force the diversity.

How was the graph created?

Steps are nodes, embedded as "approach | step". Sequential edges link step i to i+1; semantic edges link steps of different solutions at cosine ≥ 0.85.

How do you prevent test leakage?

Frozen per-family split; the bank build fails on any test id; retrieval drops same-id and same-text entries. None of it can be disabled.

Q&A · USING MEMORY

Short answers, part 2

How does retrieval work?

C/D: embed the test problem, take the 3 most similar bank entries. F: filter by family, then walk the step graph into one template of up to 8 steps.

How is it injected?

As reference text in the user turn (C, D, F-user), or pre-filled inside <think> (F-TI). User turn works: 81.2%. TI fails: 16.7%.

What does it cost?

+205 to +546 prompt tokens per question; student latency 18–30 s vs 19 s baseline. About 48 MB on disk. Building memory is 720 teacher calls, once.

Is it retrieval, or any example?

Retrieval: a shuffled template of identical shape scores 75.0% vs 81.2% retrieved. Random memory (B) gives +0.0.